

EXPERIMENT 3

Aim

To create a Cryptocurrency Using Python and Perform Mining in Blockchain.

Theory

Blockchain

A blockchain is a distributed digital ledger that stores data in a sequence of blocks. Each block contains transaction information, a timestamp, a proof value, and the hash of the previous block.

The major components of a block in this experiment are:

- **Index** – Position of the block in the blockchain.
- **Timestamp** – Time at which the block was created.
- **Transactions** – Transactions included in the block.
- **Proof** – Number obtained through the Proof-of-Work process.
- **Previous Hash** – Hash of the previous block.

Each block is linked to the previous block using its hash. Therefore, changing the contents of a previous block would invalidate the subsequent blocks.

Cryptocurrency

A cryptocurrency is a digital currency that uses cryptographic techniques and blockchain technology to record transactions without requiring a central authority.

In this experiment, a simple cryptocurrency called **HadCoin** is implemented using Python.

A transaction contains:

- Sender
- Receiver
- Amount

For example:

```
{  
  "sender": "Rutuja",  
  "receiver": "Afnan",  
  "amount": 50  
}
```

Transactions

When a transaction is submitted, it is not immediately stored inside a block. It is first stored in the node's pending transaction list:

```
self.transactions
```

When a miner mines a new block, these pending transactions are included in the block.

Mempool

A mempool is a temporary pool containing valid transactions that have been received by a blockchain node but have not yet been included in a block.

In this implementation:

```
self.transactions
```

acts as the pending transaction pool or simplified mempool.

After the transactions are included in a block, the list is cleared using:

```
self.transactions = []
```

Thus, the same transactions are not included repeatedly in subsequent blocks.

Mining

Mining is the process of creating a new block by solving a computational Proof-of-Work problem.

The program calculates:

```
hashlib.sha256(
```

```
str(new_proof**2 - previous_proof**2).encode()  
)hexdigest()
```

The mining process continues until the generated hash starts with:

0000

The proof that satisfies this condition is used to create the new block.

Mining Reward

The miner receives a reward for successfully mining a block.

In this implementation, the mining function adds a transaction similar to:

```
blockchain.add_transaction(  
    sender="Mining Reward",  
    receiver=node_address,  
    amount=1  
)
```

Therefore, a reward of **1 HadCoin** is included as a transaction in the mined block.

Multi-Node Blockchain

Three independent blockchain nodes are created.

Each node maintains its own copy of the blockchain. The nodes can communicate and maintain different versions of the blockchain until a consensus mechanism is applied.

Consensus and Longest Chain Rule

Different nodes may temporarily have different versions of the blockchain.

The consensus mechanism compares the available blockchain chains, checks their validity, and replaces the current chain if a longer valid chain is found.

Thus, the network eventually agrees on the **longest valid blockchain**.

Implementation and Output:

Step 1: Import Libraries

Step 2: Create the Cryptocurrency Blockchain

```
import datetime
import hashlib
import json
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []

        # Create Genesis Block
        self.create_block(proof=1, previous_hash='0')
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions.copy()
        }
        # Clear pending transactions after adding them to block
        self.transactions = []
        self.chain.append(block)
        return block
    def get_previous_block(self):
        return self.chain[-1]
    def proof_of_work(self, previous_proof):
        new_proof = 1
        while True:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
            # Mining condition
            if hash_operation[:4] == '0000':
                return new_proof
            new_proof += 1
    def hash(self, block):
        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()
        return hashlib.sha256(encoded_block).hexdigest()
    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        })
        return self.get_previous_block()['index'] + 1
    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            # Check previous hash
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
```

```

        ).hexdigest()
        # Check Proof of Work
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True
# Create HadCoin Blockchain
blockchain = Blockchain()
print("HadCoin Blockchain Created Successfully!")
print(blockchain.chain)

```

```

... HadCoin Blockchain Created Successfully!
[{'index': 1, 'timestamp': '2026-09-02 07:07:16.492763', 'proof': 1, 'previous_hash': '0', 'transactions': []}]

```

Step 3: Add Your Own Cryptocurrency Transaction

```

▶ sender = input("Enter sender name: ")
receiver = input("Enter receiver name: ")
amount = float(input("Enter HadCoin amount: "))

block_index = blockchain.add_transaction(
    sender,
    receiver,
    amount
)

print("\nTransaction added successfully!")
print("Transaction will be added to Block:", block_index)

```

```

... Enter sender name: Rutuja
Enter receiver name: Raje
Enter HadCoin amount: 10000

Transaction added successfully!
Transaction will be added to Block: 2

```

Step 4: Display Pending Transactions (Mempool)

```

print("Pending Transactions (Mempool):")
print(blockchain.transactions)

```

```

Pending Transactions (Mempool):
[{'sender': 'Rutuja', 'receiver': 'Raje', 'amount': 10000.0}]

```

Step 5: Mine a Block and Give Mining Reward

```

▶ node_address = "Rutuja_Miner"
previous_block = blockchain.get_previous_block()
previous_proof = previous_block['proof']
print("Mining started... Please wait.")
proof = blockchain.proof_of_work(previous_proof)
previous_hash = blockchain.hash(previous_block)
# Mining reward
blockchain.add_transaction(
    sender="Mining Reward",
    receiver=node_address,
    amount=1
)
# Create new block
block = blockchain.create_block(
    proof,
    previous_hash
)
print("\nBlock Mined Successfully!")
print(block)

```

```

... Mining started... Please wait.

Block Mined Successfully!
{'index': 2, 'timestamp': '2026-09-02 07:09:57.269494', 'proof': 533, 'previous_hash': '8f6ae557c1427adff2217e12491f6893d409f7fd06963991740f868c4c186d74',

'transactions': [{'sender': 'Rutuja', 'receiver': 'Raje', 'amount': 10000.0}, {'sender': 'Mining Reward', 'receiver': 'Rutuja_Miner', 'amount': 1}]}

```

Step 6: Verify That Pending Transactions Are Cleared

```

print("Pending Transactions after Mining:")
print(blockchain.transactions)

Pending Transactions after Mining:
[]

```

Step 7: Display the Complete Blockchain

```

print("Complete HadCoin Blockchain:\n")

for block in blockchain.chain:
    print(block)
    print("-" * 80)

Complete HadCoin Blockchain:

{'index': 1, 'timestamp': '2026-09-02 07:07:16.492763', 'proof': 1, 'previous_hash': '0', 'transactions': []}
-----
{'index': 2, 'timestamp': '2026-09-02 07:09:57.269494', 'proof': 533, 'previous_hash': '8f6ae557c1427adff2217e12491
-----

```

Step 8: Validate the Blockchain

```

valid = blockchain.is_chain_valid(blockchain.chain)

if valid:
    print("All good. The HadCoin Blockchain is valid!")
else:
    print("Blockchain is invalid!")

All good. The HadCoin Blockchain is valid!

```

Step 9: Demonstrate Multiple Nodes and Longest Chain Consensus

```

import copy

node1 = Blockchain()
node2 = Blockchain()
node3 = Blockchain()

# Give Node 1 the blockchain containing our transaction
node1.chain = copy.deepcopy(blockchain.chain)

# Node 2 receives the same chain
node2.chain = copy.deepcopy(blockchain.chain)

# Node 3 has the longest valid chain
node3.chain = copy.deepcopy(blockchain.chain)

print("Node 1 Chain Length:", len(node1.chain))
print("Node 2 Chain Length:", len(node2.chain))
print("Node 3 Chain Length:", len(node3.chain))

... Node 1 Chain Length: 2
Node 2 Chain Length: 2
Node 3 Chain Length: 2

```

```

▶ previous_block = node3.get_previous_block()

proof = node3.proof_of_work(previous_block['proof'])

previous_hash = node3.hash(previous_block)

node3.add_transaction(
    sender="Rutuja",
    receiver="Node3_User",
    amount=10
)

node3.create_block(
    proof,
    previous_hash
)

print("Updated Node 3 Chain Length:", len(node3.chain))

```

```

... Updated Node 3 Chain Length: 3

```

```

▶ nodes = [node1, node2, node3]

longest_chain = max(
    nodes,
    key=lambda node: len(node.chain)
).chain

for node in [node1, node2]:
    if len(longest_chain) > len(node.chain):
        node.chain = copy.deepcopy(longest_chain)

print("\nAfter Longest Chain Consensus:")
print("Node 1 Chain Length:", len(node1.chain))
print("Node 2 Chain Length:", len(node2.chain))
print("Node 3 Chain Length:", len(node3.chain))

```

```

...
After Longest Chain Consensus:
Node 1 Chain Length: 3
Node 2 Chain Length: 3
Node 3 Chain Length: 3

```

Conclusion

The experiment successfully demonstrated the implementation of a simple cryptocurrency and blockchain using Python. Cryptocurrency transactions were added and stored as pending transactions before mining. Proof-of-Work was performed to mine a new block, and the miner received a reward of **1 HadCoin**. The pending transaction list was cleared after the transactions were included in a block, preventing already processed transactions from being included repeatedly. The experiment also demonstrated the concept of multiple blockchain nodes and the longest-chain consensus mechanism, where nodes can synchronize with the longest valid blockchain. Thus, the experiment provided a practical understanding of **blockchain, cryptocurrency transactions, mining, Proof-of-Work, mempool, mining rewards, and consensus mechanisms**.

